

Deep Reinforcement Learning-Based Mobile Robot Navigation in Unknown Environments

1st Benpeng Huang

*College of Computer Science
and Technology,*

*Shanghai University of Electric Power
Shanghai, China*

bphuang@mail.shiep.edu.cn

2nd Jianliang Mao*

*College of Automation Engineering,
Shanghai University of Electric Power
Shanghai, China*

*Corresponding author: jl_mao@shiep.edu.cn

3rd Chuanlin Zhang

*College of Computer Science
and Technology,*

*Shanghai University of Electric Power
Shanghai, China*

clzhang@shiep.edu.cn

Abstract—Efficient and safe autonomous navigation of mobile robots in dynamic and unknown environments remains a central challenge in robotic navigation. Most traditional path-planning methods rely on prior map information and perform poorly in complex, map-less scenarios with multiple moving obstacles and randomized goal positions. Although reinforcement learning can partially address these issues, it still suffers from sparse rewards and low sample efficiency. In this paper, we propose a hybrid navigation framework that integrates velocity obstacles (VO) priors with soft actor-critic (SAC) reinforcement learning to enhance training sample efficiency and convergence speed. At the same time, we leverage the dynamic window approach (DWA) to assist in reward function design, alleviating sparse-reward problems, and employ curriculum learning to guide the agent from simple to complex tasks. We validate our approach in the PyBullet simulation environment, and results show that, compared to both traditional reinforcement learning and classical path-planning algorithms, our method delivers significant improvements in safety, efficiency, and robustness.

Index Terms—deep reinforcement learning, robot navigation, obstacle avoidance

I. INTRODUCTION

Deep reinforcement learning (DRL) has been widely applied in news recommendation, face recognition, image detection, and task decision-making. In robotics, DRL is also extensively used for localization, perception, decision-making, planning, and hardware control [1]. In the field of intelligent systems, DRL has emerged as a powerful tool for achieving complex decision making and path planning tasks [2]. Its core advantage lies in learning optimal policies through interaction with the environment, without requiring predefined rules or models [3]. This capability has demonstrated great potential and application value in autonomous driving, robot navigation, and path planning in complex environments. With the rapid advances in deep learning, the application of DRL in path planning continues to expand, offering new perspectives and methods for intelligent system design.

Early representative work introduced an asynchronous variant of the deep deterministic policy gradient (DDPG) algorithm [4] to train mapless motion planners in simulation, and successfully transferred the resulting policy to real-world

scenarios [5]. Subsequently, more stable algorithms such as proximal policy optimization (PPO) [6] and soft actor-critic (SAC) have been applied to mapless navigation tasks. In particular, SAC has demonstrated superior performance in continuous control problems, making it well-suited for mobile robot navigation [7], [8].

One significant drawback of DRL methods is their low sample efficiency. Agents typically require extensive interaction with the environment to learn the desired behaviors through trial and error [9]. In contrast, curriculum learning improves sample efficiency by progressively increasing the difficulty of training tasks, from simple to complex, allowing the agent to receive meaningful feedback earlier and reduce ineffective exploration during the initial learning stages. Researchers have applied curriculum learning to help agents first acquire basic navigation and obstacle avoidance skills in static environments before transitioning to dynamic scenarios for more robust policy development [10]. Enhancements such as task-level curricula, trajectory energy-based prioritization, and fuzzy logic controllers have further improved learning efficiency and navigation safety, ensuring reliable performance in complex dynamic environments [11]. However, curriculum learning alone may struggle in highly dynamic settings with diverse obstacle motion patterns, where limited reasoning ability impairs sustained safe and efficient navigation [12]. To address this, recent work integrates Transformer architectures into the SAC framework, incorporates periodic-review-based curriculum strategies, and optimizes LiDAR clustering to improve environmental inference, speed up training, and bridge the sim-to-real gap [13]. Another study have proposed a novel DRL-based approach that incorporates both crowd and static obstacle information, designs a safety-aware reward function, and builds a ROS-based multi-sensor navigation system, significantly improving mapless navigation performance in simulated environments [14]. However, this also increases computational complexity to some extent, which may pose challenges for real-time deployment on resource-constrained embedded platforms.

To address the above challenges, we propose an enhanced DRL framework that combines velocity obstacles (VO) [15] with SAC for mapless navigation of mobile robots. First,

This work was supported in part by the National Natural Science Foundation of China under Grant 62203292.

we employ the VO model online to project the policy's proposed velocity into a safe, feasible region, substantially reducing collision risk during early training. Second, we fuse locally optimal trajectories and velocity costs generated by the DWA into a multi-stage, dynamically weighted dense reward function, guiding the agent smoothly from "mere feasibility" to "optimal performance." Finally, we accelerate convergence via a curriculum learning strategy that gradually increases both the number and speed range of dynamic obstacles. Experimental validation in the PyBullet simulator confirms that our method significantly improves safety, efficiency, and robustness compared to baseline DRL and classical planning approaches.

II. BACKGROUND

A. Velocity Obstacle

VO is a geometric method for dynamic collision avoidance, widely adopted in multi-agent and mobile robot navigation tasks. It characterizes the set of velocities that would result in a collision between the navigating agent and a moving obstacle within a finite time horizon. As illustrated in Fig. 1, we consider two agents: robot A , with current position P_A and velocity $\mathbf{v}_A$, and a dynamic obstacle B , with position P_B and velocity $\mathbf{v}_B$. The objective is to determine whether A 's current or intended velocity $\mathbf{v}_A$ will lead to a collision with B , assuming both agents maintain their velocities.

The obstacle in the configuration space $B \oplus (-A)$, denoted by the Minkowski sum of the shape of the obstacle B and the negated shape of agent A , is used to define the collision region. The shaded cone-shaped region represents the $VO_B^A(\mathbf{v}_B)$, which includes all velocities of A that would result in a collision with B , assuming B moves at velocity $\mathbf{v}_B$.

Mathematically, the area is defined as:

$$VO_B^A(\mathbf{v}_B) = \{\mathbf{v}_A \mid (\mathbf{v}_A - \mathbf{v}_B) \in \mathcal{C}\} \quad (1)$$

where $\mathcal{C}$ is the collision cone generated by the relative positions and radii of A and B . In this formulation, $\mathbf{v}_A - \mathbf{v}_B$ is the relative velocity between agent A and obstacle B , and $\mathcal{C}$ is the set of all relative velocities that would bring the two agents into contact. In Fig. 1, the triangle formed by vectors $\mathbf{v}_A$, $\mathbf{v}_B$, and $\mathbf{v}_A - \mathbf{v}_B$ geometrically illustrates the relative motion. If the vector $\mathbf{v}_A - \mathbf{v}_B$ lies within the shaded VO cone, a collision is imminent unless the trajectory is modified. The apex of the VO cone is located at the origin of the velocity space, and its axis aligns with the vector from P_A to P_B . The boundaries of the cone are tangents to the combined radius circle of A and B , effectively representing the reach of their possible contact region.

To avoid collision, the velocity $\mathbf{v}_A$ must be selected outside of $VO_B^A(\mathbf{v}_B)$. Therefore, a safe velocity can be computed as:

$$\mathbf{v}^* = \arg \min_{\mathbf{v} \notin VO_B^A(\mathbf{v}_B)} \|\mathbf{v} - \mathbf{v}_{\text{pref}}\|^2 \quad (2)$$

where $\mathbf{v}_{\text{pref}}$ denotes the preferred velocity (e.g., toward the goal), and $\mathbf{v}^*$ is the closest admissible velocity that ensures collision avoidance. This formulation permits online projection

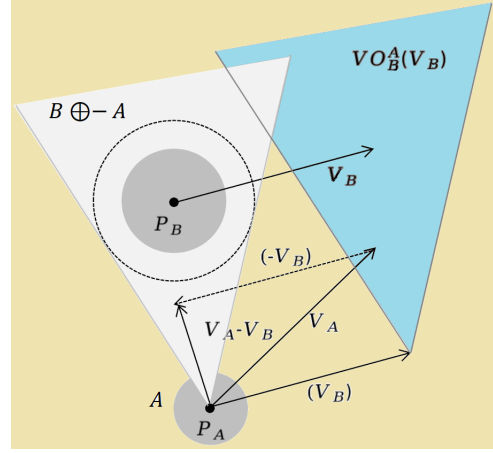


Fig. 1. Principle of VO.

of an unsafe candidate action $\mathbf{v}_{\text{pref}}$ into the safe set, which is especially beneficial during early learning stages in reinforcement learning when the agent tends to generate unsafe trajectories.

In summary, the VO framework provides a geometrically intuitive and computationally efficient method to incorporate dynamic obstacle avoidance into policy optimization, by constraining the agent's velocity selections within a safe region in velocity space.

B. Soft Actor-Critic

SAC is a leading off-policy DRL algorithm that trades off exploration and exploitation by augmenting the reward with an entropy bonus [16]. SAC maintains two soft Q-functions $Q_{\omega_1}, Q_{\omega_2}$ and a stochastic policy π_θ (the "actor"), as well as slowly updated target networks $\bar{Q}_{\omega_1}, \bar{Q}_{\omega_2}$ for stabilization.

At each update, transitions (s_t, a_t, r_t, s_{t+1}) are sampled from a replay buffer $\mathcal{D}$. The critic networks are trained by minimizing the soft Bellman residual:

$$L(\omega_i) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}} \left[\frac{1}{2} (Q_{\omega_i}(s_t, a_t) - y_t)^2 \right] \quad (3)$$

where the target is defined as:

$$y_t = r_t + \gamma \mathbb{E}_{a' \sim \pi_\theta(\cdot | s_{t+1})} \left[\min_{j=1,2} \bar{Q}_{\omega_j}(s_{t+1}, a') - \alpha \log \pi_\theta(a' | s_{t+1}) \right] \quad (4)$$

The actor network is updated to maximize expected return plus entropy:

$$L(\theta) = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_\theta} \left[\alpha \log \pi_\theta(a_t | s_t) - \min_{j=1,2} Q_{\omega_j}(s_t, a_t) \right] \quad (5)$$

Here, α is a temperature parameter that weights the entropy term and may be tuned or learned automatically. Actions are sampled using the reparameterization trick from a Gaussian

policy, which outputs mean and log-std. After each gradient step, the target networks are softly updated as:

$$\bar{\omega}_j \leftarrow \tau \omega_j + (1 - \tau) \bar{\omega}_j, \quad j = 1, 2 \quad (6)$$

with $\tau \ll 1$. This combination of entropy regularized objectives, double Q-learning, and target smoothing enables SAC to achieve both sample efficiency and robust exploration in continuous control tasks.

III. METHOD

This section provides a detailed overview of the optimization algorithms and their application to map-less mobile robot navigation.

A. Problem Formulation

We model navigation as a finite-horizon markov decision process (MDP) (S, A, P, r, γ) . At each step the agent observes its pose (x, y, θ) , a fixed goal location, and the positions and velocities of M moving obstacles. Actions are continuous linear and angular velocities (v, ω) , which under differential-drive kinematics update the robot's pose.

To ensure safety, we first compute the nominal velocity vector

$$\mathbf{v}^{\text{cmd}} = (v \cos \theta, v \sin \theta) \quad (7)$$

and then filter it through a Velocity-Obstacle (VO) safety layer: any commanded velocity falling into the union of obstacle-induced collision cones is projected back into the safe velocity set.

Finally, we learn a policy $\pi_\theta(a | s)$ via SAC to maximize the expected discounted return

$$J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right] \quad (8)$$

By layering deep RL on top of VO-based filtering, the agent achieves both robust dynamic-obstacle avoidance and efficient goal reaching.

B. Action Space and State Space

In this study, the action space is designed as a two-dimensional continuous vector $\mathbf{a}_t = [v_t, \omega_t]$, where the linear velocity $v_t \in (0, 1)$ and angular velocity $\omega_t \in (-1, 1)$ represent the robot's translational and rotational motion control commands, respectively. The state space integrates environmental perception and dynamic historical information, containing the current relative goal position $\mathbf{s}_t^g$, the robot's real-time velocity $\mathbf{s}_t^v$, 32-dimensional LiDAR scan data $\mathbf{s}_t^{\text{lidar}}$, and incorporates 10-dimensional historical information including the previous timestep's robot velocity and partial state features. This forms a compact 46-dimensional representation, a design that avoids information redundancy by selecting key environmental features and motion history, significantly improving the model's learning efficiency and real-time decision-making capability.

C. Reward Function

A well-designed reward function can accelerate the agent's learning of target behaviors. This paper proposes a reward mechanism for dynamic dense environments. When the agent is in state s_t and executes action a_t at time t , the environmental reward is defined as:

$$r(s_t, a_t) = \begin{cases} r_{\text{arrive}} & \text{if arrival} \\ -r_{\text{penalty}} & \text{if collision / timeout} \\ r_{\text{DWA}} & \text{otherwise} \end{cases} \quad (9)$$

When the robot reaches the goal, it receives a positive reward r_{arrive} and the episode terminates. When the robot collides, it receives a negative reward r_{penalty} and the episode also terminates. The navigation reward is designed based on the evaluation function of DWA, as follows:

$$r_{\text{DWA}} = w_{\text{prog}} r_{\text{prog}} + w_{\text{head}} r_{\text{head}} + r_{\text{surv}} + w_{\text{obs}} r_{\text{obs}} \quad (10)$$

where r_{surv} is a constant survival reward. The other reward components are defined as:

$$\begin{cases} r_{\text{prog}} = d_{t-1} - d_t \\ r_{\text{head}} = \cos(\theta_t - \phi_t) \\ r_{\text{obs}} = -\frac{1}{d_{\min} + \varepsilon} \end{cases} \quad (11)$$

Here, d_t denotes the Euclidean distance between the robot's current position and the goal at time step t , and r_{prog} is the progress reward for moving toward the goal. θ_t is the robot's current heading, and ϕ_t is the direction of the vector pointing to the goal. The cosine of their difference measures the alignment between heading and goal direction. r_{obs} is the obstacle-avoidance reward based on obstacle distance, where $d_{\min}$ is the minimum distance from the robot's current position to all obstacles. To prevent division by zero, $\varepsilon > 0$ is added. The weights w_{prog} , w_{head} , and w_{obs} adjust each reward component and can be dynamically tuned during training.

D. Network Structures

Our navigation framework relies on two primary neural networks: an actor network that parameterizes the stochastic policy and a pair of critic networks that approximate the soft Q-functions. The actor network accepts a 46-dimensional state vector - comprising the 32-dimensional LiDAR scan, the 2-dimensional relative goal position, and the 12-dimensional previous velocity and partial state - and processes it through two fully connected hidden layers of 256 units each, using ReLU activations to introduce non-linearity. Its output layer then produces a two-dimensional mean vector and a corresponding 2-dimensional log-standard deviation vector of a Gaussian distribution, from which the continuous linear and angular velocity commands are sampled. Each critic network takes as input the same 46-dimensional state concatenated with the 2-dimensional action and passes them through two ReLU-activated hidden layers of 256 units, finally yielding a scalar Q-value estimate. By maintaining two critics, we mitigate overestimation bias by taking the minimum predicted Q value during optimization. All hidden layers use ReLU nonlinearity,

while the actor's output layers use linear activation for the mean and softplus for the log-standard deviation to ensure numerical stability.

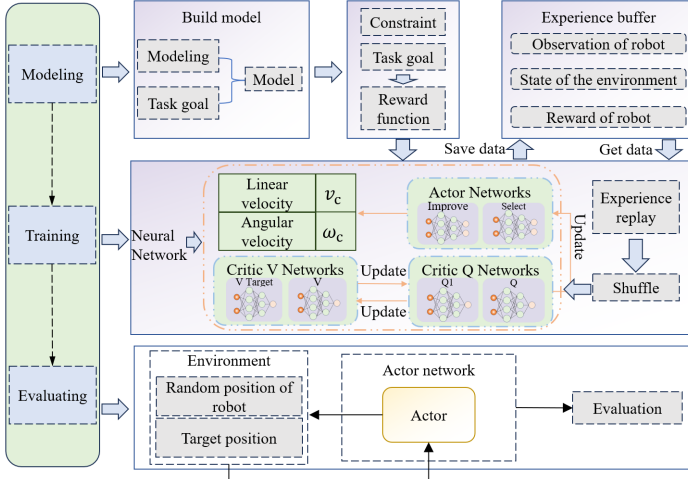


Fig. 2. The proposed algorithm.

E. Learning Procedure

The algorithm first formulates the robot navigation task as a MDP, where the environment states and continuous actions define the state and action spaces, respectively. In terms of network architecture, it employs a twin Q-network to estimate the “soft” value function, along with a Gaussian policy network that outputs the action distribution via the reparameterization trick. During training, an experience replay buffer is used to store the agent’s interactions with the environment, and three key updates are performed iteratively: minimizing the soft Bellman error to optimize the Q-networks, maximizing the entropy-regularized policy objective to update the policy network, and adaptively adjusting the temperature parameter to balance reward and entropy. In the decision-making phase, the agent samples actions from the policy network, achieving efficient and robust navigation in complex and unknown environments, as illustrated in Fig. 2.

IV. PERFORMANCE VALIDATION

Our simulation environment is built on the PyBullet physics engine and features a TurtleBot robot alongside multiple dynamic obstacles. The robot is instantiated from the TurtleBot3 URDF and operates as a differential-drive vehicle on a 2D plane. Obstacles, modeled as cubes or cylinders, are spawned at random positions, with their number, speed, and spatial distribution sampled according to a predefined curriculum-based difficulty schedule. The scene uses a fixed-height plane bounded by surrounding walls, with gravity set to -10 m/s^2 and a simulation time step of $\Delta t = 0.1 \text{ s}$. Robot and obstacle velocities are initialized via the `resetBaseVelocity` API and advanced each tick by calling `stepSimulation`. To evaluate navigation performance, we conduct repeated trials across varying obstacle densities (1–6), speed ranges (0–0.3 m/s), and

goal distances (3–8 m), recording metrics such as cumulative reward, success rate (SR), and average navigation time, as shown in Fig. 3.

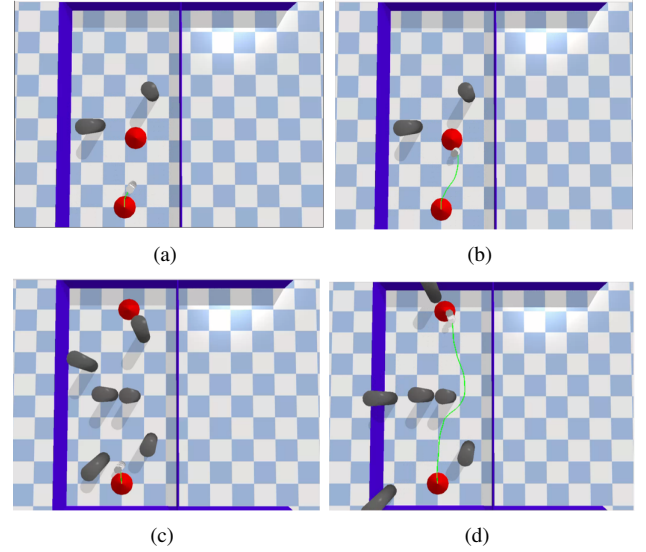


Fig. 3. (a)-(b) show trajectory plots in the PyBullet simulation environment under simple tasks in the early stage of training. (c)-(d) show trajectory plots in the PyBullet simulation environment under complex tasks in the later stage of training.

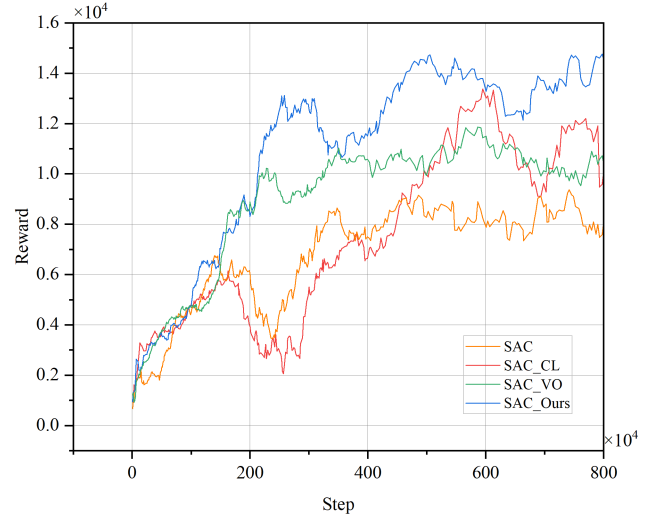


Fig. 4. The impact of different optimization methods on model performance.

First, we compare the impact of three optimization methods on model performance: SAC_CL (the SAC algorithm without VO), SAC_VO (the SAC algorithm without curriculum learning), and SAC_Ours (the SAC algorithm with VO and curriculum learning). As shown in Fig 4, our hybrid algorithm not only quickly suppressed inefficient exploration in the early stages but also maintained high stability in the later stages, achieving a higher mean reward than the other three methods. Next, we combined our optimization strategy with three mainstream reinforcement learning algorithms (TD3,

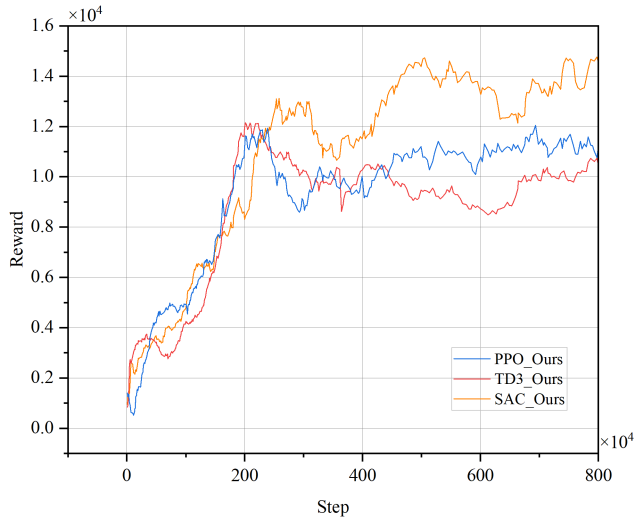


Fig. 5. The impact of optimization methods combined with different reinforcement learning algorithms on model performance.

PPO, SAC) and evaluated their performance in the same environment. The results, presented in Fig 5, indicate that the optimization strategy paired with SAC yields the best overall performance, with cumulative rewards surpassing those of the other algorithms.

To verify that the incorporation of VO projection and curriculum learning enhances decision quality and reduces the risk of the robot oscillating in place or getting stuck in local optima, we compared the navigation performance of four optimization strategies in the same experimental scenario. As shown in Table 1, our method, which combines VO projection with curriculum-based reward shaping, achieves a success rate above 80%, exceeding each baseline of each strategy by more than 10%. Moreover, it also reduces the average navigation time, demonstrating that the synergy of these two techniques can safely guide the agent to quickly learn efficient paths.

TABLE I
PERFORMANCE COMPARISON OF ALGORITHMS

	SR	Navigation time(s)
SAC	0.429	23.32
SAC_CL	0.688	19.78
SAC_VO	0.712	21.46
SAC_Ours	0.836	18.34

V. CONCLUSION

This paper proposes a hybrid navigation framework that integrates VO, SAC, and curriculum learning strategies. In terms of the reward function, a dense reward formulation is designed based on the DWA, enabling efficient and safe autonomous navigation for mobile robots in complex scenarios with multiple dynamic obstacles, no prior map, and randomized goals. Simulation results show that, compared with traditional reinforcement learning methods, the proposed method achieves improvements in success rate, average navigation

time, and cumulative rewards, while effectively alleviating issues of sparse rewards and low sample efficiency. Future work will explore deployment and validation on real robotic platforms, and incorporate more sensor fusion and multi-agent cooperation to further enhance the algorithm's generalization and practical value.

REFERENCES

- [1] Q. Liu, J. Mao, L. Han, C. Zhang, and J. Yang, "Predictive observer-based dual-rate prescribed performance control for visual servoing of robot manipulators with view constraints," *IEEE Transactions on Cybernetics*, vol. 55, no. 5, pp. 2424–2436, 2025.
- [2] W. He, Z. Jiang, T. Xiao, Z. Xu, and Y. Li, "A survey on uncertainty quantification methods for deep learning," *arXiv preprint arXiv:2302.13425*, 2023.
- [3] Y. Zhang, M. Zhu, Y. Gong, and R. Ding, "Optimizing science question ranking through model and retrieval-augmented generation," *International Journal of Computer Science and Information Technology*, vol. 1, no. 1, pp. 124–130, 2023.
- [4] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.
- [5] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 31–36.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [7] W. Zhang, Y. Zhang, N. Liu, K. Ren, and P. Wang, "Ipaprec: A promising tool for learning high-performance mapless navigation skills with deep reinforcement learning," *IEEE/ASME Transactions on Mechatronics*, vol. 27, no. 6, pp. 5451–5461, 2022.
- [8] K. Bektaş and H. I. Bozma, "Apf-rl: Safe mapless navigation in unknown environments," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 7299–7305.
- [9] T. Zhu, J. Mao, L. Han, C. Zhang, and J. Yang, "Real-time dynamic obstacle avoidance for robot manipulators based on cascaded nonlinear mpc with artificial potential field," *IEEE Transactions on Industrial Electronics*, vol. 71, no. 7, pp. 7424–7434, 2024.
- [10] W. Yu, J. Peng, Q. Qiu, H. Wang, L. Zhang, and J. Ji, "Pathrl: An end-to-end path generation method for collision avoidance via deep reinforcement learning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 9278–9284.
- [11] Y. Yin, Z. Chen, G. Liu, J. Yin, and J. Guo, "Autonomous navigation of mobile robots in unknown environments using off-policy reinforcement learning with curriculum learning," *Expert Systems with Applications*, vol. 247, p. 123202, 2024.
- [12] J. Mao, W. Song, H. Liang, F. Xia, and C. Zhang, "Graph-optimized encoder-imu fusion for robust pipeline robot localization in confined spaces," *IEEE Transactions on Instrumentation and Measurement*, vol. 74, pp. 1–12, 2025.
- [13] C. Yang, S. Bi, Y. Xu, and X. Zhang, "Ctsac: Curriculum-based transformer soft actor-critic for goal-oriented robot exploration," *arXiv preprint arXiv:2503.14254*, 2025.
- [14] B. Xue, F. Zhou, C. Wang, M. Gao, and L. Yin, "Robot mapless navigation in vuca environments via deep reinforcement learning," *IEEE Transactions on Industrial Electronics*, 2024.
- [15] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *The international journal of robotics research*, vol. 17, no. 7, pp. 760–772, 1998.
- [16] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*. Pmlr, 2018, pp. 1861–1870.